SFTP User Management Guide

User Management Guide — Accounts, Permissions, Keys

Revision: 1 **Last modified:** 2026-07-11T15:23:48Z

Full account lifecycle for the enterprise SFTP system: create, update, disable, delete, list — through the REST API, the web admin console, and the mobile clients. Permission model, super-admin bootstrap, password policy, and SSH key authentication included.

API endpoints marked **PLANNED — lands with STREAM-2 (ATM-002)** / console screens **PLANNED — STREAM-4 (ATM-004)** / mobile **PLANNED — STREAM-5 (ATM-005)** reflect the committed design in `docs/Issues.md`; they are not yet executable.

Table of contents

1. Concepts
2. Super-admin bootstrap
3. Creating accounts (API)
4. Creating accounts (web console)
5. Creating accounts (mobile)
6. Permission model
7. The public flag — never default
8. Listing and inspecting accounts
9. Updating accounts
10. Deleting and disabling accounts
11. Password policy
12. SSH key authentication
13. What happens under the hood
14. Related docs

1. Concepts

- **Account** — an SFTP login identity with a home directory under the data root, a permission level, and optional SSH keys. Accounts are stored in the system DB and rendered to `users.conf` (atmoz/sftp format) — the DB is authoritative; `users.conf` is a generated artifact.
- **Super-admin** — the only role that can manage accounts and view the audit log. Exactly one is created at bootstrap; additional super-admins are PLANNED post-MVP.
- **users.conf line grammar** (atmoz/sftp, verified 2026-07-11):

  `user:pass[:e][:uid[:gid[:dir1[,dir2]...]]]`

  `:e` marks the password field as an encrypted hash. The renderer always writes hashes, never plaintext.

2. Super-admin bootstrap

During `bash scripts/setup.sh` (ATM-006) the API (ATM-002) creates the super-admin once:

```
# PLANNED — ATM-002/006
curl -fsS -X POST http://127.0.0.1:7722/api/v1/bootstrap \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"<prompted-by-setup.sh-never-
        logged>"}'
```

Bootstrap is single-shot: a second call returns `409 Conflict`. The password is prompted interactively by `setup.sh` and never echoed or logged (§11.4.10).

3. Creating accounts (API)

```
# PLANNED — ATM-002. Obtain a token first:
TOKEN=$(curl -fsS -X POST http://127.0.0.1:7722/api/v1/auth/login
        \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"<super-admin-password>"}' |
        jq -r .token)

curl -fsS -X POST http://127.0.0.1:7722/api/v1/accounts \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "alice",
    "password": "<strong-password>",
    "permission": "read_write",
```

```
    "public": false
  }'
```

Field rules (enforced by the API, ATM-002/003):

Field	Required	Rule
`username`	yes	lowercase, `[a-z0-9_.-]`, unique
`password`	conditional	required unless `ssh_key` supplied; policy in §11
`permission`	yes	`read_only` or `read_write` — no other values
`public`	no	default `false`; `true` requires `public_acknowledged: true` (§7)
`ssh_key`	no	OpenSSH public key line; appended to the user's `authorized_keys`

4. Creating accounts (web console)

PLANNED — STREAM-4 (ATM-004): Dashboard → Accounts → **New account**. The form mirrors the API contract: username, password (strength meter), permission radio (`read_only` / `read_write`), and a **Public** switch that stays off until the operator ticks the explicit acknowledgement checkbox. Light/dark themes via OpenDesign tokens; English i18n first.

5. Creating accounts (mobile)

PLANNED — STREAM-5 (ATM-005): KMP clients (Android/iOS/HarmonyOS/AuroraOS) share the same API client. Flow: sign in as super-admin → Accounts → **+** → same fields as the web form. Same `public` acknowledgement guard — no mobile shortcut around it.

6. Permission model

Permission	SFTP capabilities	Enforcement points
`read_only`	list, download	API validation → renderer writes

Permission	SFTP capabilities	Enforcement points
		account with read-only enforcement → filesystem: upload/ rename/delete rejected (write-mask); container round-trip evidence: RO upload **denied** (ATM-003 acceptance)
`read_write`	list, download, upload, rename, delete (own home)	API validation → renderer → directory provisioner grants write under the user's home subtree only

Detailed enum + enforcement map: ../architecture/permissions_model.md.

7. The public flag — never default

`public: true` makes an account reachable without authentication safeguards appropriate for anonymous access. The constitution-level rule (operator mandate, ATM-002):

- Default is **always `false`** — in the DB schema, the API, the web form, and the mobile form.
- Setting `true` requires an **explicit acknowledgement** in the same request (`public_acknowledged: true`) or the equivalent console checkbox.
- Every public enablement writes an audit-log entry (actor, timestamp, account).
- A public account is always `read_only` — `public: true` + `read_write` is rejected at validation.

8. Listing and inspecting accounts

```
# PLANNED — ATM-002
curl -fsS -H "Authorization: Bearer $TOKEN" http://127.0.0.1:7722/
        api/v1/accounts | jq
curl -fsS -H "Authorization: Bearer $TOKEN" http://127.0.0.1:7722/
        api/v1/accounts/alice | jq
```

Responses never include password material — only `username`, `permission`, `public`, `has_ssh_key`, `created_at`, `last_login_at`, `status`.

9. Updating accounts

```
# PLANNED — ATM-002: rotate password
curl -fsS -X PATCH http://127.0.0.1:7722/api/v1/accounts/alice \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/
        json' \
  -d '{"password":"<new-strong-password>"}'

# Change permission
curl -fsS -X PATCH http://127.0.0.1:7722/api/v1/accounts/alice \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/
        json' \
  -d '{"permission":"read_only"}'
```

Every update re-renders `users.conf` atomically and signals the SFTP container to reload (§13). In-flight sessions are not killed; new logins use the new credentials immediately.

10. Deleting and disabling accounts

```
# PLANNED — ATM-002
curl -fsS -X DELETE http://127.0.0.1:7722/api/v1/accounts/alice \
  -H "Authorization: Bearer $TOKEN"            # deletes account;
        data retained by default
curl -fsS -X DELETE 'http://127.0.0.1:7722/api/v1/accounts/alice?
        purge_data=true' \
  -H "Authorization: Bearer $TOKEN"            # also removes the
        home directory — audit-logged
```

- Deleting removes the account from the DB and `users.conf`; the home directory is **retained** unless `purge_data=true` (data-safety default, §9).
- Disabling (`PATCH {"status":"disabled"}`) blocks login without touching data — preferred for temporary suspensions.
- The last super-admin cannot delete or demote itself.

11. Password policy

Enforced by the API (ATM-002), surfaced in the web/mobile forms:

- Minimum length 12; must not equal the username; must not appear in a common-password blocklist.
- Stored only as a hash: API DB (Argon2id), `users.conf` (`crypt` hash with `:e` marker — atmoz/sftp supports encrypted entries, verified 2026-07-11).

- Plaintext never appears in logs, audit entries, or API responses (§11.4.10).
- Rotation via §9; forced-rotation-on-first-login is PLANNED post-MVP.

12. SSH key authentication

Preferred over passwords for automation and production access.

```
# PLANNED — ATM-002: attach a key to an account
curl -fsS -X POST http://127.0.0.1:7722/api/v1/accounts/alice/
        keys \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/
        json' \
  -d '{"public_key":"ssh-ed25519 AAAA… alice@laptop"}'
```

Mechanics (atmoz/sftp, verified 2026-07-11): public keys are mounted into the user's `.ssh/keys/` directory and the container appends them to `.ssh/authorized_keys` (direct `authorized_keys` bind-mounts fail OpenSSH permission checks). A key-only account has an empty password field (`user::uid:gid`). Client side:

```
sftp -P 7721 -i ~/.ssh/id_ed25519 alice@<server-ip>
```

13. What happens under the hood

Account create/update/delete flows (ATM-002/003 design):

1. API validates the request (schema + permission enum + public-guard).
2. DB transaction commits the account row (SQLite dev / PostgreSQL prod).
3. The renderer regenerates `users.conf` atomically (write-temp-then-rename) from the DB — no half-written lines.
4. The directory provisioner creates/repairs the user's home + writable subdirectory under the data root.
5. The API signals the SFTP container to reload (recreate on config change) via the containers submodule — never `docker/sudo`.
6. An audit-log row records actor, action, account, timestamp.
7. Next SFTP login uses the new state.

14. Related docs

- security_guide.md — secrets, hardening, audit
- deployment_guide.md — install + verification
- troubleshooting_guide.md — auth failures, permission-denied
- ../architecture/permissions_model.md

- ../tutorials/api_quickstart.md — full curl walkthrough

Sources verified (2026-07-11)

- Internal: `docs/Issues.md` ATM-002/003/004/005 (API contract, permission enum, public-guard, screens) · `docs/research/mvp/MVP.md` (users.conf baseline) · `docs/plans/master_implementation_plan.md`.
- External fetched this revision: `https://github.com/atmoz/sftp` — users.conf grammar, `:e` encrypted marker, `.ssh/keys/` mount semantics, key-only accounts (`user::uid`).
- To re-verify before each release (§11.4.99): atmoz/sftp README.